Part 2: The Somewhat Homomorphic Scheme (SHE)

Speaker Notes & Mathematical Primer

Nirav Bhattad

March 27, 2026

§1. The Symmetric Foundation

How to start speaking: "We are now going to build a Fully Homomorphic Encryption scheme from scratch. To do this, we won't start with the complex public-key version. Instead, we are going to look at a symmetric-key version[cite: 17]. It is beautifully simple, and if you understand this symmetric version, the rest of the paper will fall perfectly into place."

Before defining the algorithms, we establish the parameters based on a security parameter λ:

- η: The bit-length of the secret key.
- ρ: The bit-length of the noise.

The symmetric scheme is defined by three algorithms:

- $\mathsf{KeyGen}(\lambda)$: Choose a random odd integer p from the interval $[2^{\eta-1}, 2^{\eta})$. The integer p is the secret key.
- $\mathsf{Encrypt}(p, m)$: To encrypt a bit $m \in \{0, 1\}$, we choose a large random integer q and a small random integer r (where $|r| < 2^{\rho}$). We compute the ciphertext c as:

$$c = p \cdot q + 2r + m \tag{1.1}$$

- $\mathsf{Decrypt}(p, c)$: To recover the message, we compute:

$$m' = (c \bmod p) \bmod 2 \tag{1.2}$$

Walking through the Decryption: "Let's pause and look at that decryption equation, because it is the engine of the entire paper. Our ciphertext is $c = pq + 2r + m$. What happens when we take this modulo p?
Because pq is an exact multiple of p, it vanishes. We are left with the remainder: $2r + m$. But wait—the modulo operation only returns $2r + m$ if this noise term is strictly less than $p/2$ in absolute value! If the noise grows too large, it "wraps around" the modulus p, and decryption fails.
Assuming the noise is small enough, $(c \bmod p)$ gives us exactly $2r + m$. Now, what happens when we take $(2r + m) \bmod 2$? The $2r$ is an even number, so it vanishes, leaving only our original plaintext bit m. Simple, elegant, and completely reliant on the noise staying small."

§2. Transitioning to a Public-Key Scheme

A symmetric scheme is useful, but a true FHE scheme must allow anyone to encrypt data We transform our symmetric scheme into a public-key scheme using a technique that relies on publishing a list of "encryptions of zero."
We introduce two new parameters:

- γ: The bit-length of the integers in the public key.
- τ: The number of integers in the public key.
- $\mathsf{KeyGen}(\lambda)$: The secret key remains the odd integer p. To create the public key, we generate $\tau + 1$ samples of the form $x_i = p \cdot q_i + 2r_i$. We ensure x_0 is the largest element, x_0 is odd, and its noise term r_0 is even. The public key is $pk = \langle x_0, x_1, \ldots, x_\tau \rangle$.
- $\mathsf{Encrypt}(pk, m)$: To encrypt a bit $m \in \{0, 1\}$, we choose a random subset $S \subseteq \{1, 2, \ldots, \tau\}$ and a small random noise integer r. The ciphertext is:

$$c = \left[m + 2r + 2 \sum_{i \in S} x_i \right]_{x_0} \tag{2.1}$$

Intuition for the Public Key: "Why does this work? Every x_i in the public key is essentially an encryption of zero. By summing a random subset of these x_i values, we are adding $0 + 0 + 0 \ldots$ together, generating a brand new, randomized encryption of zero. We then add our message bit m to it. The modulo x_0 step at the end is just to keep the physical size of the ciphertext from growing infinitely large, without destroying the underlying modulo p structure."

§3. Homomorphism and The Noise Bottleneck

We now have a working public-key encryption scheme. But is it homomorphic? To answer this, we must define the Evaluate algorithm. In this scheme, Evaluate is beautifully trivial: it simply applies integer addition and multiplication directly to the ciphertexts.
Let's observe what happens when we operate on two ciphertexts, $c_1 = pq_1 + 2r_1 + m_1$ and $c_2 = pq_2 + 2r_2 + m_2$.

3.1. Homomorphic Addition

When we add the two ciphertexts, we group the terms:

$$c_1 + c_2 = p(q_1 + q_2) + 2(r_1 + r_2) + (m_1 + m_2) \tag{3.1}$$

Notice the structure: we have a multiple of p, plus $2 \times$ (new noise), plus the sum of the messages. The new noise is exactly $(r_1 + r_2)$. By the triangle inequality, addition only increases the noise linearly.

3.2. Homomorphic Multiplication

Now, let us multiply the ciphertexts:

$$\begin{aligned} c_1 \cdot c_2 &= (pq_1 + 2r_1 + m_1)(pq_2 + 2r_2 + m_2) && (3.2) \\ &= p(pq_1q_2 + q_1(2r_2 + m_2) + q_2(2r_1 + m_1)) \\ &\quad + 2(2r_1r_2 + r_1m_2 + r_2m_1) + m_1m_2 && (3.3) \end{aligned}$$

Again, the structure is preserved! We have a massive multiple of p, plus 2 times a mixed noise term, plus the product of the messages $m_1 m_2$.

The Climax of Part 2: The Noise Bottleneck "Look closely at the noise term in the multiplication equation: $2r_1r_2$. When we multiply two ciphertexts, the noise doesn't just add; it multiplies. It squares the magnitude of the noise.
Remember our strict decryption rule? $(c \bmod p)$ only works if the accumulated noise is strictly less than $p/2$. Because multiplication squares the noise, we can only perform a very limited number of sequential multiplications before the noise term explodes past $p/2$, destroying the plaintext.
This is why the scheme is currently only **Somewhat Homomorphic**. It can evaluate any circuit, but only up to a very specific multiplicative depth. In Part 4, we will solve this using Gentry's Bootstrapping Theorem."